

Databricks Asset Bundles

A standard, unified approach to
deploying data products on
Databricks

Agenda

1

Databricks Asset Bundles: DAB

2

Demo: Infobase use case

3

Call to Action

Anatomy of your projects in Databricks

Let's describe them

Consist of a variety of components

Code: Notebooks, Python .whl, JAR, dbt, etc.

Execution Environment:

Databricks Workspace, compute configuration

Resources: Databricks Workflows, MLflow Tracking Server and Registry, Delta Live Tables...

Produce a variety of data products

Create tables and pipelines, machine learning models, dashboards, call external services, etc.

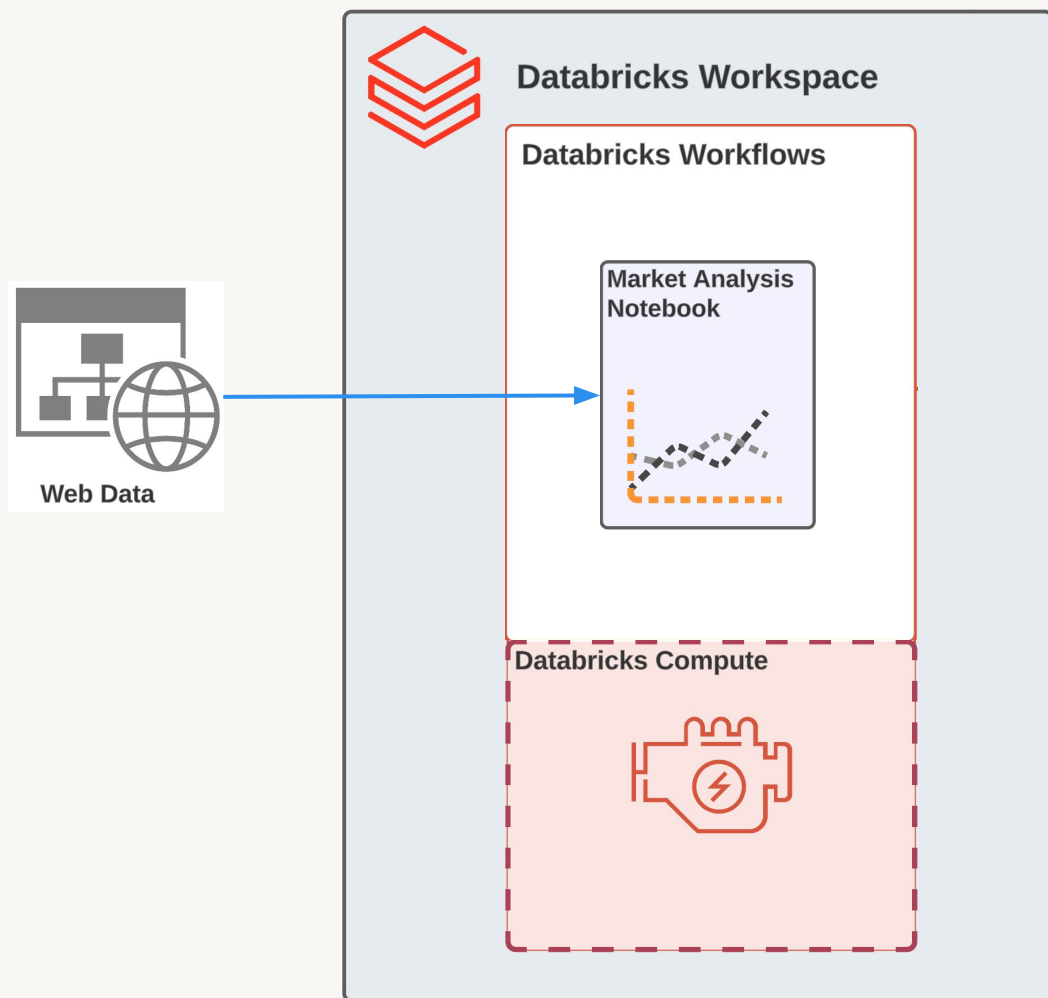
The task determines the components

A simple report might consist of a notebook running on single node compute

A full MLOps pipeline would require MLflow, Feature Store, and Model Serving components



Let's break this down further...



Data product: Weekly market trends report

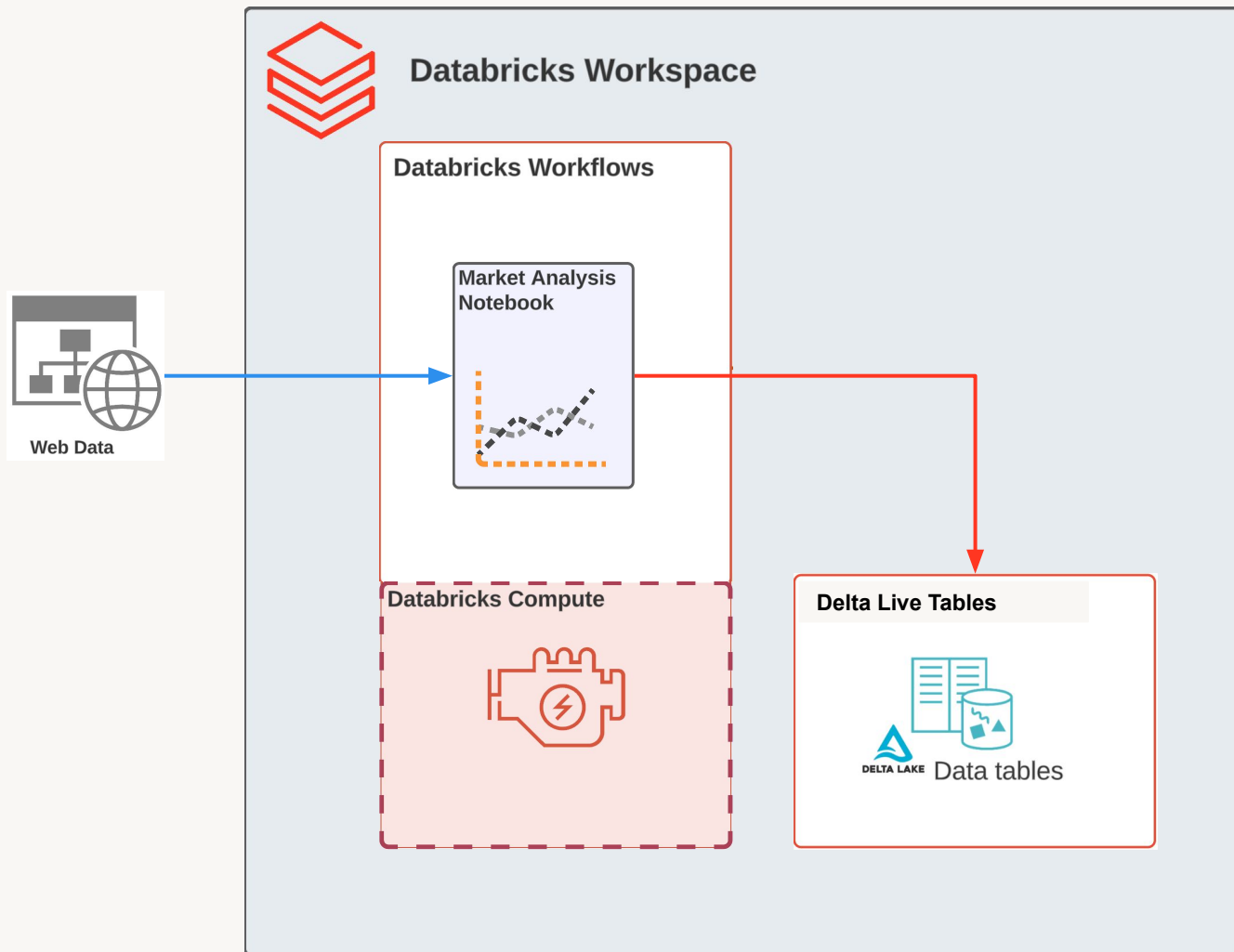
Code: Python/R Notebook

Resources: Databricks Workflow

Execution Environment:
Databricks Workspace, compute configuration

What if our task changes?

Just add the necessary components to complete the task



Data product: Weekly market trends report *+persisted table*

Code: Python/R/SQL Notebook

Resources: Databricks Workflow, *+Delta Live Tables*

Execution Environment:
Databricks Workspace,
compute configuration

In other words...

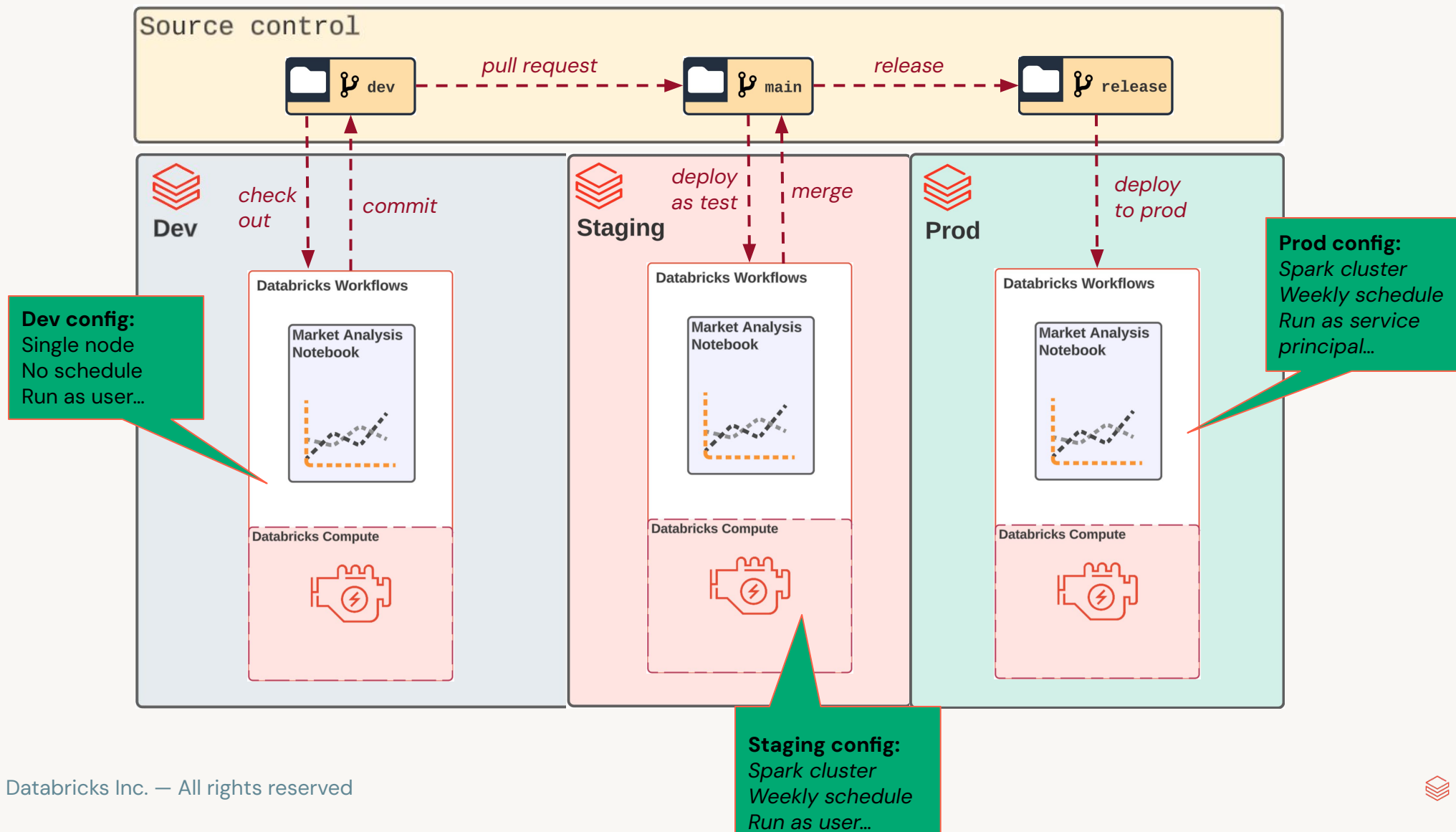
Your project consists of:
code + resources + env



How do we get your project into production?



The journey to production



How do we orchestrate this journey?

Getting to production is challenging

Many ways to do it today, but none ideal

Use infrastructure as code with Databricks TF Provider

- Powerful, general platform automation
- Perfect for DevOps rocket scientists, tough for everyone else

Use dbx by Databricks Labs

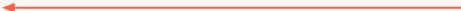
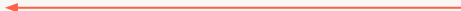
- More accessible than Terraform, simpler than using REST APIs directly
- No formal support for Databricks Labs projects

Use Databricks REST APIs directly

- Low level
- Scriptable, DIY
- Error prone, brittle – generally not recommended

Steps to deploy Databricks projects

Programmatically, with the REST API

1. Where do you want to run it?  Workspace APIs
2. What do you want to run?
 - a. Code artifacts: Notebooks, Python packages or JAR files.  Workspace, Jobs APIs
 - b. Workflow tasks, dependencies and configuration  Clusters, MLflow APIs
 - c. Workspace resources: DLT, MLflow experiment, etc.  Jobs, Clusters APIs
3. Go run it!
 - a. Upload application code, create and configure resources  DBFS, Jobs APIs
 - b. Schedule and run your workflow  Jobs APIs

CI/CD: a solved problem, or is it?

dbx: Databricks labs project

```
lennart.kats@WWNWF20V6Q /tmp % mkdir demo  
lennart.kats@WWNWF20V6Q /tmp % c
```

👍 easy to use

👍 supports Python and Scala code

😞 not a general solution: no notebooks, only Python/Scala code

😞 Labs project

😞 not natively supported

CI/CD: a solved problem, or is it?

Terraform

- 💪 supported by eng (Deco team)
- 💪 very powerful, expressive
- 💪 general: notebooks, dashboards, service principals, ...



- 💪 admin tool of choice for configuring workspaces
- 😞 challenge for data scientists: HCL language
- 😞 challenge for data scientists: Terraform state files
- 😞 challenge for data scientists: only caters to deploys, not full dev loop

How can we simplify?

Write code once, deploy to multiple workspaces easily

- Co-version code with all configs in a simple, easy to understand format (e.g., YAML)
- Define Databricks resources using existing REST API parameters
- Specify environment-based overrides and variables
- Ensure user isolation during deployment

Implementing a standard, holistic way to deploy with configs... DABs

Introducing Databricks Asset Bundles



Databricks Asset Bundles

Where are bundles used?

Write code once, deploy everywhere

YAML files that specify the artifacts, resources, and configurations of a Databricks project.

What are Databricks Asset Bundles?

The **new databricks CLI** has functions to **validate**, **deploy and run** Databricks Asset Bundles using `bundle.yml` files

How do bundles work?

Bundles are useful during **development and CI/CD** processes

A closer look

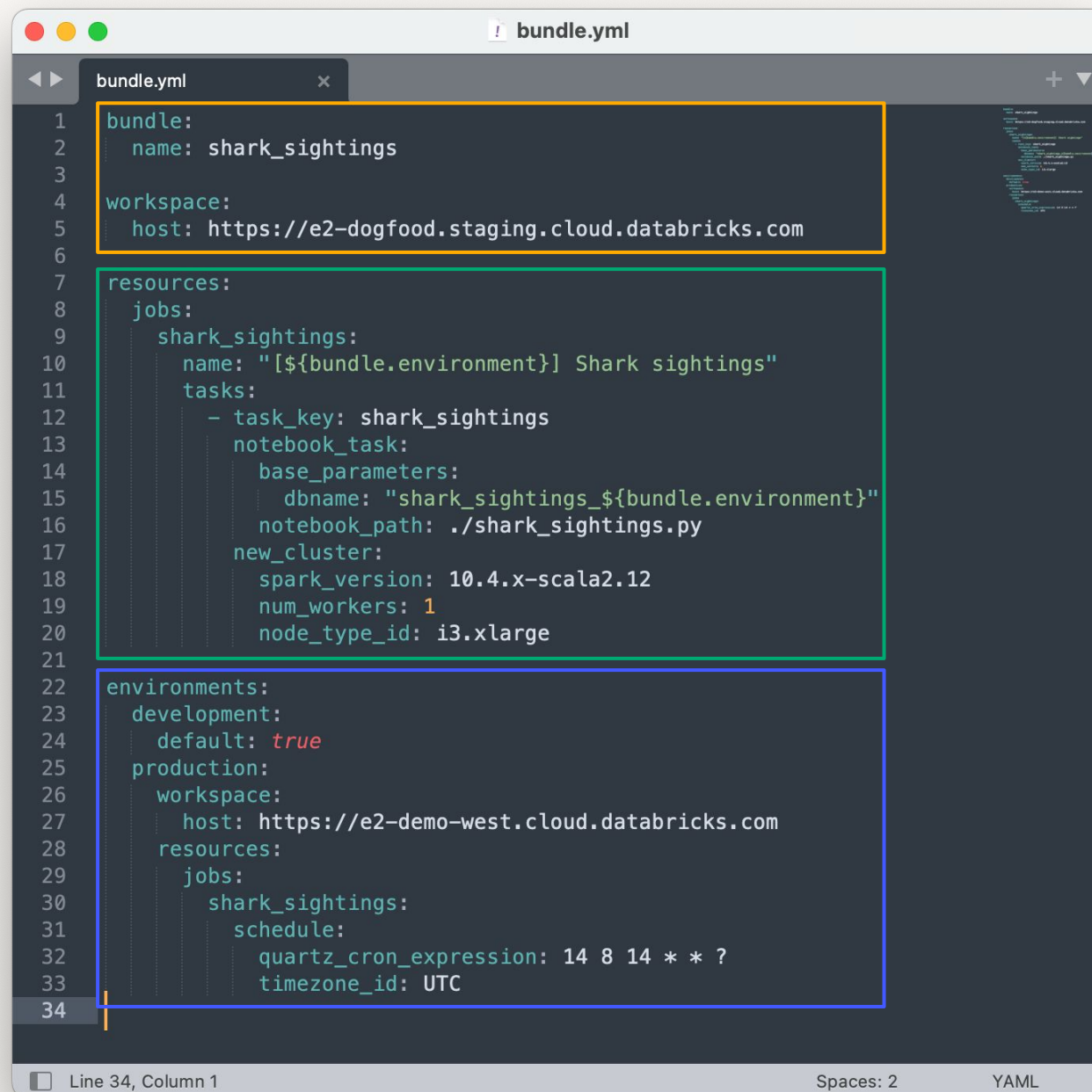
Name and default Workspace

Resource configurations

- Jobs, DLT pipelines, MLflow, etc.
- Follows REST API schema

Environment-based specs

- Control project behavior in different environments



```
bundle.yml
1 bundle:
2   name: shark_sightings
3
4 workspace:
5   host: https://e2-dogfood.staging.cloud.databricks.com
6
7 resources:
8   jobs:
9     shark_sightings:
10      name: "[${bundle.environment}] Shark sightings"
11      tasks:
12        - task_key: shark_sightings
13          notebook_task:
14            base_parameters:
15              dbname: "shark_sightings_${bundle.environment}"
16            notebook_path: ./shark_sightings.py
17          new_cluster:
18            spark_version: 10.4.x-scala2.12
19            num_workers: 1
20            node_type_id: i3.xlarge
21
22 environments:
23   development:
24     default: true
25   production:
26     workspace:
27       host: https://e2-demo-west.cloud.databricks.com
28     resources:
29       jobs:
30         shark_sightings:
31           schedule:
32             quartz_cron_expression: 14 8 14 * * ?
33             timezone_id: UTC
34
```

Line 34, Column 1 Spaces: 2 YAML

Where can we use bundles?

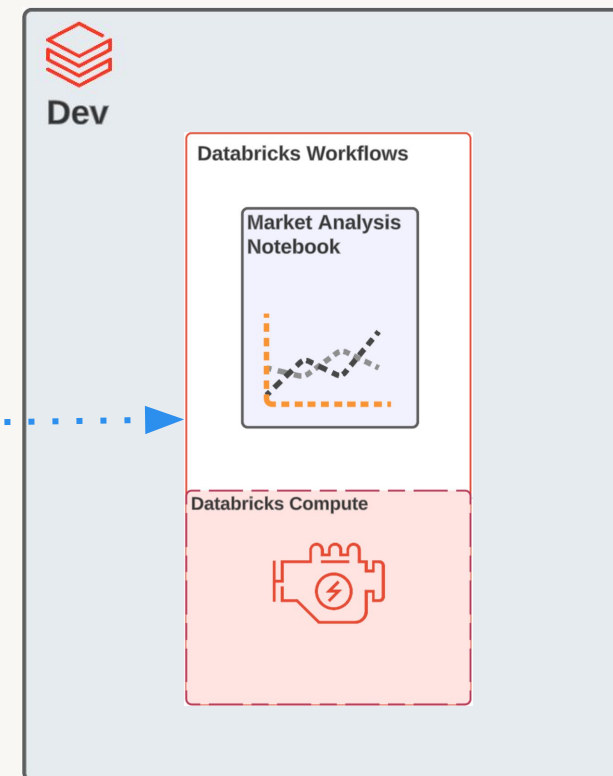
As part of active development

Alice

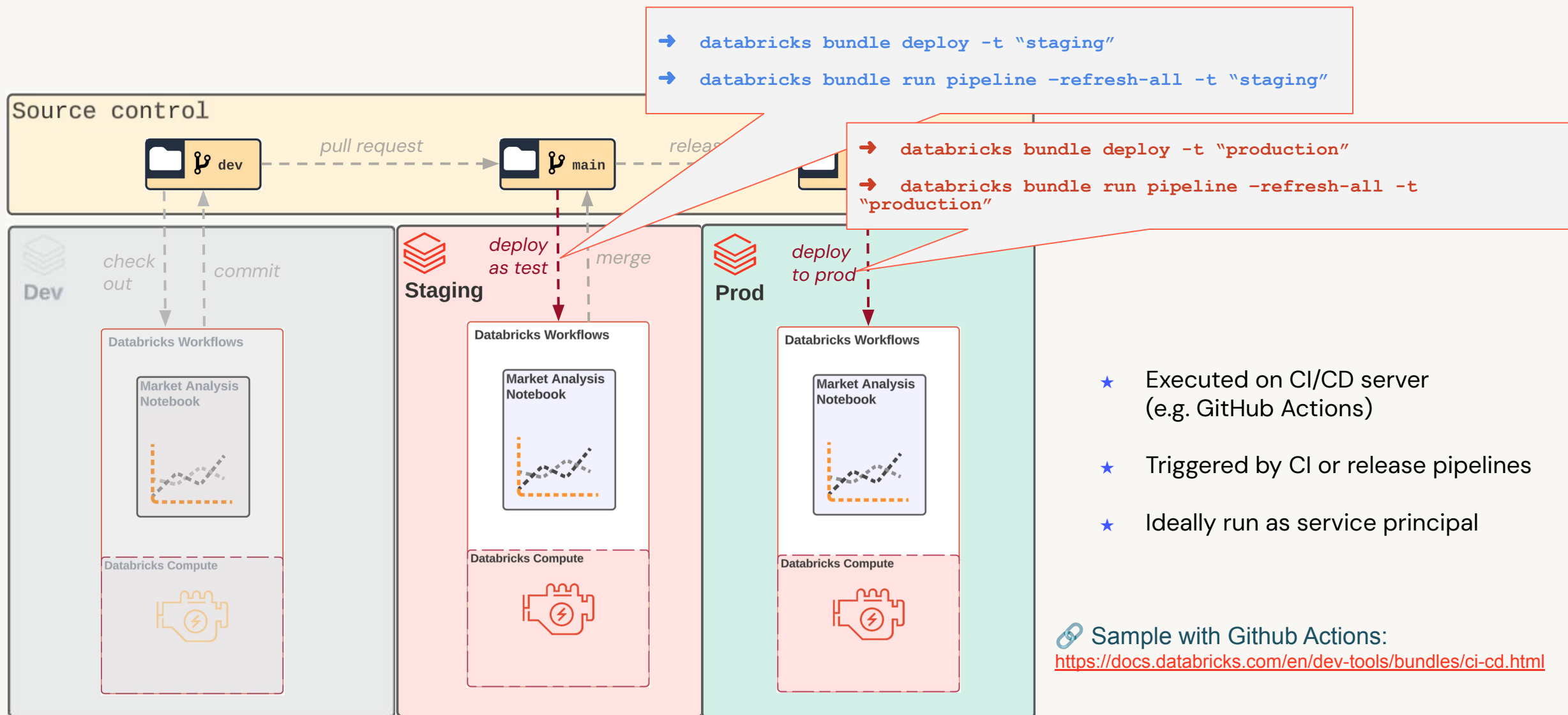


```
$ databricks bundle deploy -t "development"
$ databricks bundle run pipeline -refresh-all -t "development"
```

- ★ Deploy and run your project, tweak configs, deploy and test changes
- ★ Deploy to multiple workspaces for testing differences
- ★ Deploy and run from IDEs, terminals, or Databricks



Where can we use bundles?



- ★ Executed on CI/CD server (e.g. GitHub Actions)
- ★ Triggered by CI or release pipelines
- ★ Ideally run as service principal

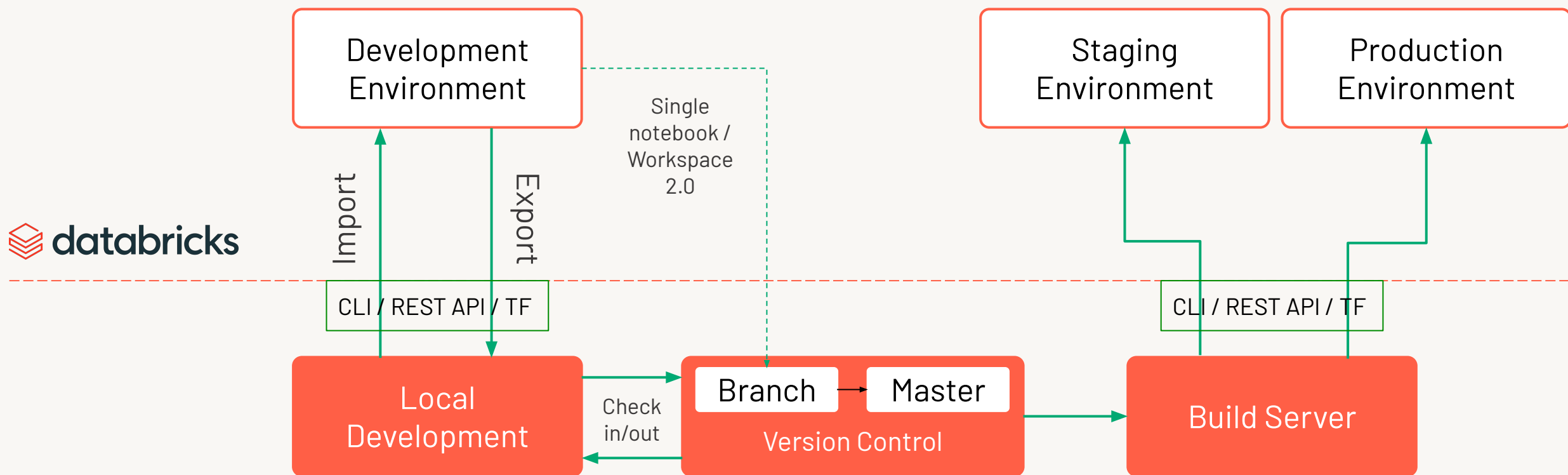
 Sample with Github Actions:
<https://docs.databricks.com/en/dev-tools/bundles/ci-cd.html>

DABs: Write code once, deploy anywhere

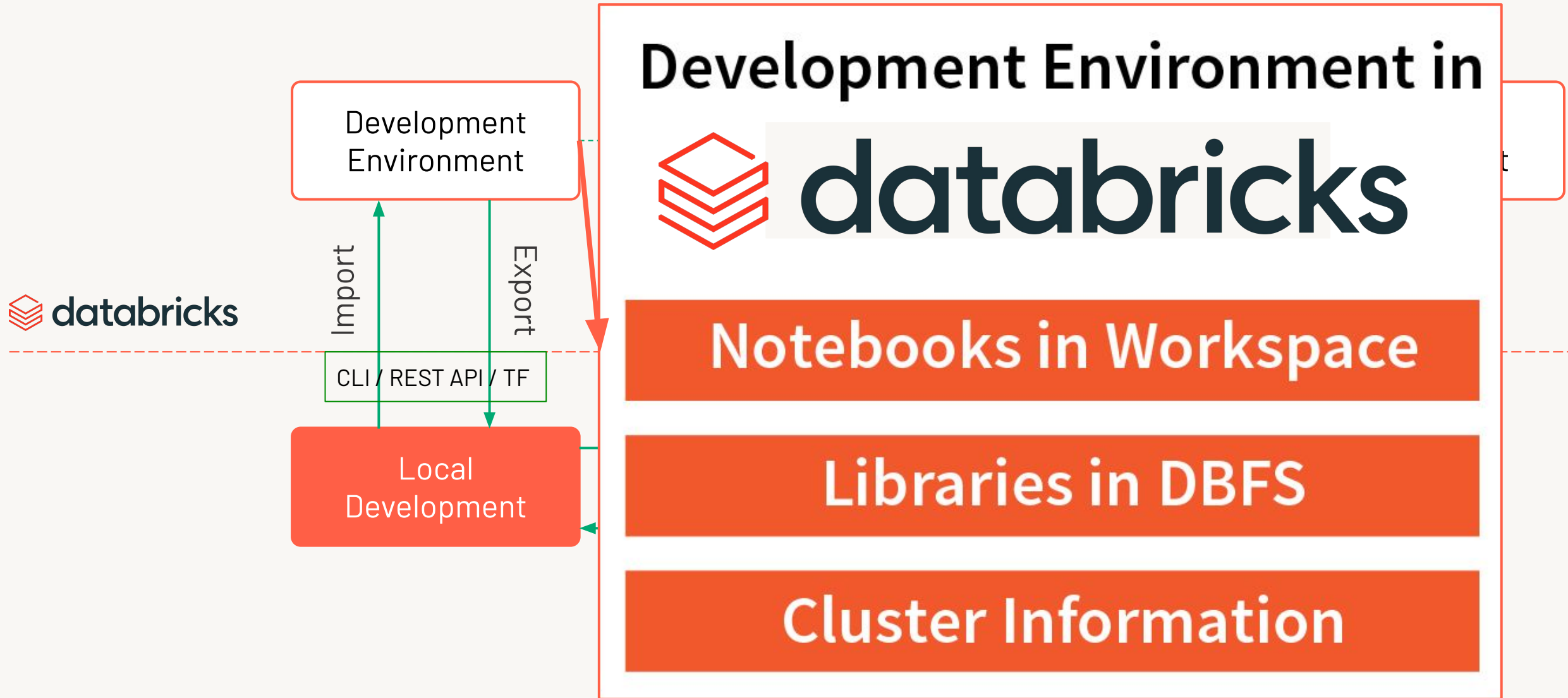
Summary

- **Democratizing best practices for CI/CD and project management**
 - Co-version configs with code and resources
 - Automation friendly (CLI vs. 1000 REST calls)
 - Collaborate and deploy with confidence (isolation, automated testing, etc)
- **Unified, single way to define and deploy your projects**

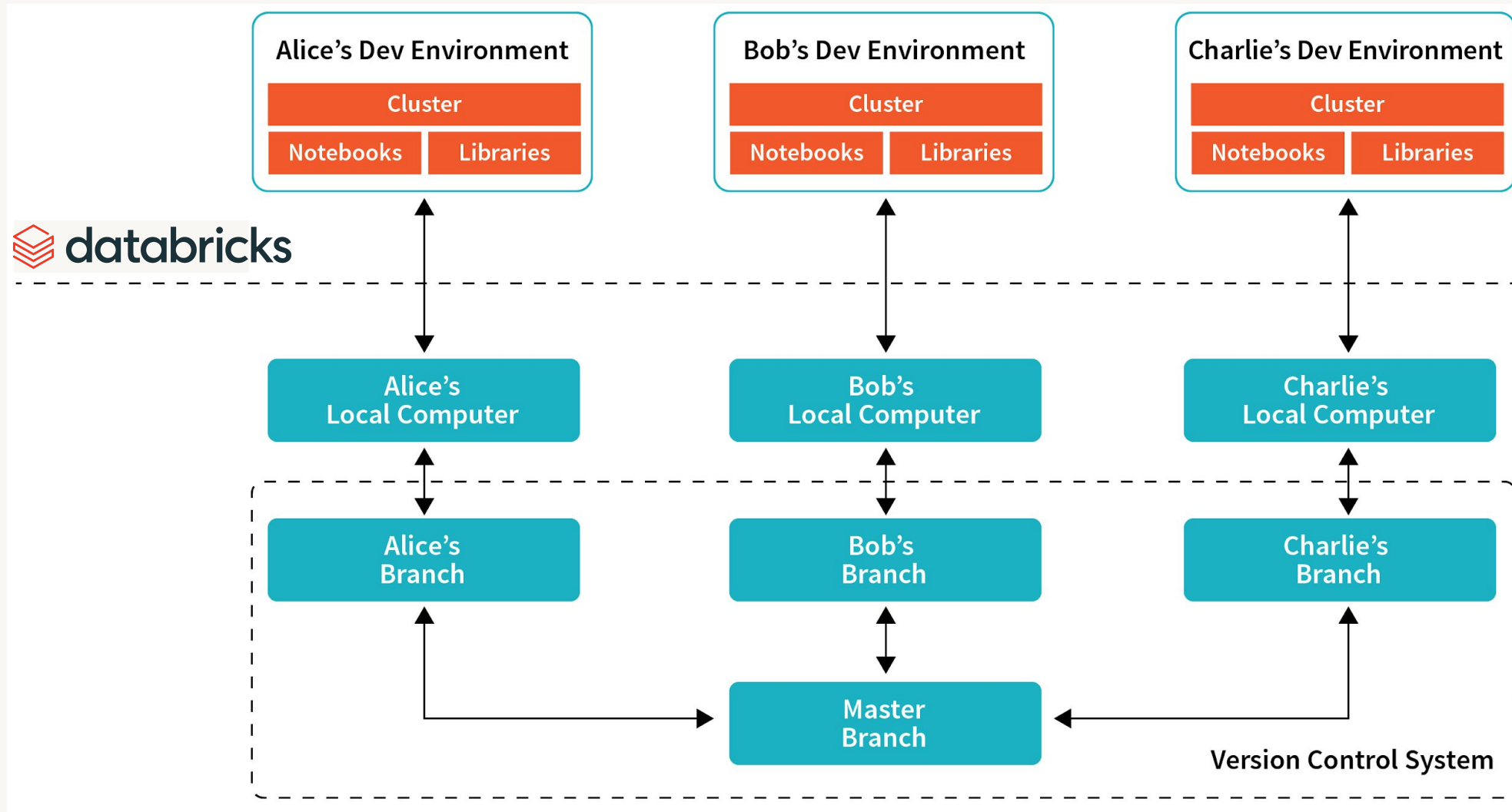
CI/CD for Notebooks: Overview



CI/CD for Notebooks: Development



CI/CD for Notebooks: Development



Call to Action

Call to Action

- Take a look at DAB walkthrough shown earlier available at:

<https://app.getreprise.com/launch/W68Ng9X/>

- Give it a try
- Raise questions